Exploring DeepFakes: Face Reenactment Report

October 12, 2026

1 Autoencoders

1.1 Introduction & Problem Definition

The goal of our project is swapping faces between two members of the group while keeping the source identity, regenarate the target video with source's mimic and movements meanwhile trying to keep the video's quality. This document will explain one of there alternative architectures: autoencoders.

We first decided to build a small auto encoder first by ourselves before trying pretrained models to experience and learn basics. We then tried pretrained face-swapping models and compared their outputs with our own approach. We covered full process, from preparing the images and training a model to generating videos and looking closely at what worked and what did not.

1.2 Dataset Preparation & Preprocessing

1.2.1 Data collection and frame extraction

We recorded short videos of Person A and Person We saved every fifth frame instead of every frame, in order to avoid collecting too many nearly identical images.

1.2.2 Face detection and image preparation

Next, we detected faces with OpenCV's Haar Cascade classifier. We converted frames to grayscale for detection. We also added a 25% margin around each face before cropping, so the images included some surrounding facial features.

Images were resized to 128 × 128 pixels. During training, PyTorch's ToTensor transformation scaled the pixel values to the 0–1

1.3 Autoencoder Architecture

We built the model using one Encoder and two separate Decoders. Decoder A was trained on Person A's images and Decoder B on Person B's. The Decoders had the same design, but they learned their own parameters.

Dataset	Prepared face images	Resolution
Person A	100	128 × 128
Person B	138	128 × 128
Total	238	128 × 128

Table: Dataset

Epoch	Person A MSE	Person B MSE
146	0.003046	0.001939
147	0.003013	0.001929
148	0.003014	0.001905
149	0.002996	0.001882
150	0.002935	0.001866

Table 1: Recorded training results

Experiment	A → B output	B → A output
50 epochs	deepfake_A_to_B.mp4	deepfake_B_to_A.mp4
150 epochs	deepfake_150_A_to_B.mp4	deepfake_150_B_to_A.mp4

Table: Experiments

The Encoder had four convolutional layers. The Decoder reversed this process with four transposed-convolution layers. We used Sigmoid activation kept the output pixel values between 0 and 1.

First, we sent encoders's representation to the Decoder for the same person to reconstruct the input face. To try face swapping, secondly, we used the other person's Decoder: A through Decoder B, and B through Decoder A.

1.4 Model Training

In each epoch, we first trained on Person A's images with Decoder A and then on Person B's images with Decoder B. For each run, the model calculated the reconstruction error, updated the shared Encoder and the relevant Decoder.

We trained for 50 epochs and saved the model weights. We then loaded those weights and continued for another 100 epochs.

At epoch 150, Person A's average training MSE was 0.002935 and Person B's was 0.001866.

1.5 Experiments & Results

1.5.1 Video generation and comparison

We generated videos after both 50 and 150 epochs. We tested A → B and B → A using generate_video.py and generate_video_150.py.

1.5.2 Observations

When we compare the videos we could not see a clear improvement between 50 and 150 epochs. In addition to that, the generated A → B face did not look like Person B, and the B → A result did not look like Person A. In other words, our model was learning reconstruction but not performing the identity transfer we wanted.

We think the main issue was the training objective. The model learned to recreate the original face instead of swapping it with another face.Our small dataset and may also have played a part, although we did not test those factors separately. This unsuccessful experiment helped us decide what to try next.

1.6 Experiments with Pretrained Face-Swapping Models

After the autoencoder experiments, we tested three pretrained models in FaceFusion: HyperSwap 1A 256, InSwapper 128 and SimSwap 256. We used a source face image and a target video,

Model	Output size	Frames	Duration
HyperSwap 1A 256	576 × 1024	576	~19.25 s
InSwapper 128	576 × 1024	576	~19.25 s
SimSwap 256	576 × 1024	576	~19.25 s

Table: Pretrained models

with the source providing the identity and the target providing the expressions and movement. We ran the models for inference and did not retrain them

HyperSwap and InSwapper looked fairly natural and gave similar results. SimSwap followed the facial movements too, but we noticed more visible differences around the eyes, the middle of the face and the skin texture.

This was a visual comparison, but it was clear that the pretrained models gave more promising results than our own autoencoder.

1.7 Evaluation & Lessons Learned

1.7.1 Overall evaluation

Training and producing videos was not successfull, however. FaceFusion gave better results with HyperSwap and InSwapper. For us, the difference showed how important it is to use a pretrained model for a spesific task.

We had only 100 images of Person A and 138 of Person B, it was a limitation.

1.7.2 Lessons learned

The biggest lesson was that a low loss does not mean a usable result. Our model was getting better in reconstructuring faces but not good at the same as swapping the faces. Training for another 100 epochs did not fix this.

As a sumary, we learned much more from the project than how to run a face-swapping tool. Building our own model helped us understand preprocessing, training and evaluation. Using pretrained models afterwards, gave us a chance to compare how it would improve the result

2 GAN & FOMM

A generator is like a painter who draws faces. A discriminator is like an expert who judges each face as real or fake and teaches the painter. This game cannot follow orders, however. For reenactment we must keep one identity and copy the motion from a video, so we used the First Order Motion Model (FOMM).

2.1 Standard GANs

A standard GAN holds two networks. The generator G reads a noise vector z and draws a face; the discriminator D reads an image and gives a verdict: real or fake. The two train against each other, so the generator learns to fool the discriminator (Goodfellow et al., 2014), as Figure 1 shows.

We cannot control the output. Identity, pose and mood are all mixed into the noise input. We cannot say “keep this person, copy that motion”. A reenactment model needs two inputs instead: a source image (who) and a driving video (how it moves).

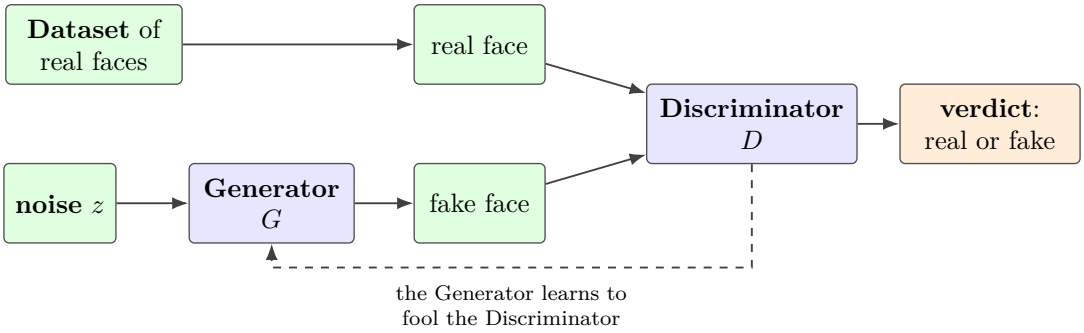


Figure 1: The training loop of a standard GAN. The Discriminator compares each fake face with real faces and gives a verdict; the dashed feedback teaches the Generator to fool it.

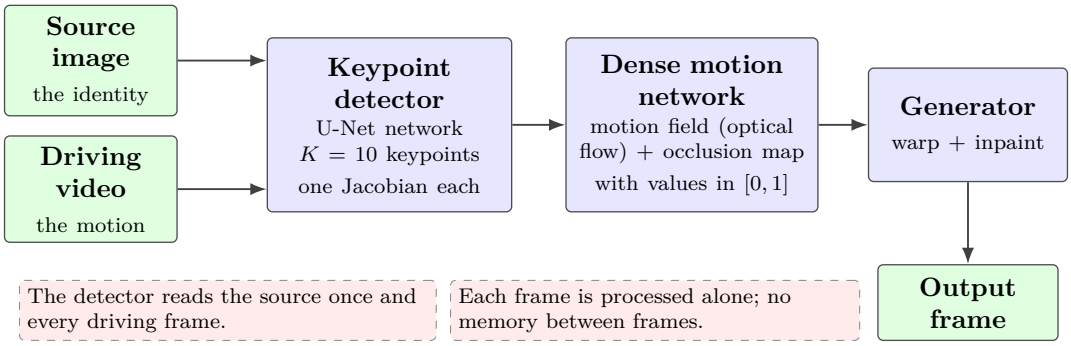


Figure 2: The First Order Motion Model pipeline: keypoints with Jacobians, a motion field and an occlusion map, then warp and inpaint. Each frame is processed alone.

2.2 First Order Motion Model

FOMM animates one source photo with the head motion and the facial expressions from a driving video. The method comes from Siarohin et al. ("First Order Motion Model for Image Animation", NeurIPS 2019). It needs no landmark detector and no 3D face model. It learns its own keypoints per object category; our category is faces. It follows Monkey-Net, which the paper calls the first object-agnostic deep model, and FOMM improves on it.

2.2.1 First-order motion

0th order motion moves a point only in x and y, so the skin keeps its shape and the result looks stiff and robotic. 1st order motion also measures how the skin around the point turns, grows and stretches, through a small 2×2 Jacobian matrix per keypoint. The paper calls it a first order Taylor expansion.

2.2.2 The three parts

The pipeline has three parts (Figure 2).

The **keypoint detector** is a U-Net network (the paper's word; the code names the class Hourglass). It finds $K = 10$ keypoints for the faces model and one Jacobian per keypoint. The expected point position is a weighted average over the output heatmaps (soft-argmax). An equivariance loss teaches the detector during training, so it needs no keypoint labels.

The **dense motion network** turns the sparse keypoints into a dense motion field (optical flow) and an occlusion map with values between 0 and 1.

The **generator** warps the source features with the flow. The code multiplies the warped features by the occlusion map (`out = out * occlusion_map`). Where the map is near 0, the

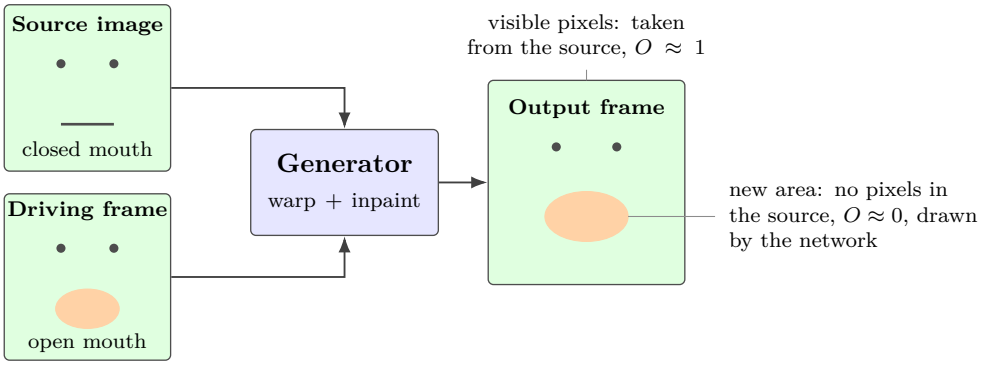


Figure 3: Occlusion and inpainting. Where O is near 1, the pixels come from the source. Where O is near 0, the network draws the missing pixels itself.

Item	Value
Checkpoint	`vox-cpk.pth.tar` (VoxCeleb)
Source portrait	512 × 512
Driving clip	256 × 256, 320 frames, 29.97 FPS
Mode and flags	CPU, `--relative`, `--adapt_scale`
Output	256 × 256, 320 frames, 29.97 FPS, 10.7 s
Speed	about 2.05 it/s, about 2.5 minutes

Table 2: The local FOMM run: setup, flags and output.

warped pixels lose their weight and the decoder fills the area from learned face priors (inpainting).

Each frame is processed on its own. No state passes from frame to frame (no recurrence).

2.2.3 Occlusion and inpainting

A closed mouth or a turned head reveals pixels that do not exist in the source photo (self-occlusion). Where O is near 1, the pixels come from the source. Where O is near 0, the network draws the area itself from the training-set average (Figure 3).

These are the failure modes we saw. Rare personal details are lost in inpainted areas. Very large head turns, about 45 degrees or more, leave blurry patches. Both limits come from our own observation, not from the paper.

2.3 Local experiment

We ran the released VoxCeleb checkpoint `vox-cpk.pth.tar` in CPU mode on an Apple Silicon machine. The source portrait is 512x512; the driving clip is 256x256, 320 frames, 29.97 FPS. With `--relative` the keypoints move relative to the first frame; `--adapt_scale` adjusts the motion size with the convex hull of the keypoints.

The output is 256x256, 320 frames, 29.97 FPS and 10.7 seconds long. The recorded speed is about 2.05 iterations per second, about 2.5 minutes in total. Head turns and blinks transferred smoothly; mouth motion needed inpainting. Table 2 lists the setup and the output.

2.4 Failure mode: braces

The VoxCeleb training data has no braces, so the mouth area should lose them when inpainting starts. Our frames confirm this (Table 3).

The active path decides which pixels appear: warped areas keep the pixels of the person, and inpainted areas follow the training data. FOMM gave the project a fast per-frame method.

Frame	Time	Driving mouth	Active path and result
10	0.3 s	small shift	warping, $O \approx 1$: braces kept
195	6.5 s	lips close	mouth hidden
288	9.6 s	wide smile	inpainting, $O \approx 0$: plain teeth

Table 3: The braces test: the active path decides where the mouth pixels come from.

Identity details survive only in warped areas, and rare details such as braces are lost as soon as inpainting starts. There is no memory across frames, so the method cannot repair a frame from its neighbours.

3 Rectified Flow Head Swap

3.1 Scope and hardware

We wanted to swap a video face with one from a selfie. The run had to stay on one laptop with a 4 GB NVIDIA GPU and 8 GB of system RAM. We sent no data to a cloud service.

The output had to keep the source motion, expression, lighting and background. Only the head should change. The clip is 96 frames at 512x512 and lasts 3.2 seconds.

3.2 How we measured

Every number comes from one script, run once per variant, each on the same 96 frames (175 to 270). The results table reports it. Face change is how much the face region moves from one frame to the next. The unswapped footage sets the level to match, so a value far below it means the smoother removed real motion too. Consecutive identity is how similar a face is to the one before it, measured in a face recognition embedding, so higher is more stable. Identity to the portrait measures similarity to the reference portrait; the unswapped footage scores near zero because it is a different person. Hair noise is how much the hair region flickers, and lower is better; p95 marks the worst frames, the value 95 percent stay under.

3.3 Pipeline overview

The flow has four stages. Generation transfers the identity, restoration cleans each face with GFPGAN, smoothing removes the boil, and reassembly attaches the audio. Only the last stage writes a video file.

The boil is the flicker that comes from generating each frame on its own. Figure 4 shows the chain in order, with the cost of each stage.

3.4 How a rectified flow model works

A rectified flow model learns one job: connect a clean image and noise with a straight line. Training picks one random time per example and mixes the two ends at that time. The model predicts the direction that points from the noise end to the image end. The guess is compared with the true direction along the line, and the weights are then nudged.

At sampling time the same model starts from pure noise. Each Euler step moves the sample a small way along the predicted direction, and an image forms. The sampling code calls these steps denoising steps, because the sample becomes less noisy at each step. Figure 5 draws training and sampling side by side.

We did not train the base model; we used a released checkpoint. The released model is step distilled, so four steps are enough for it. We could not tell the difference between four steps and eight.

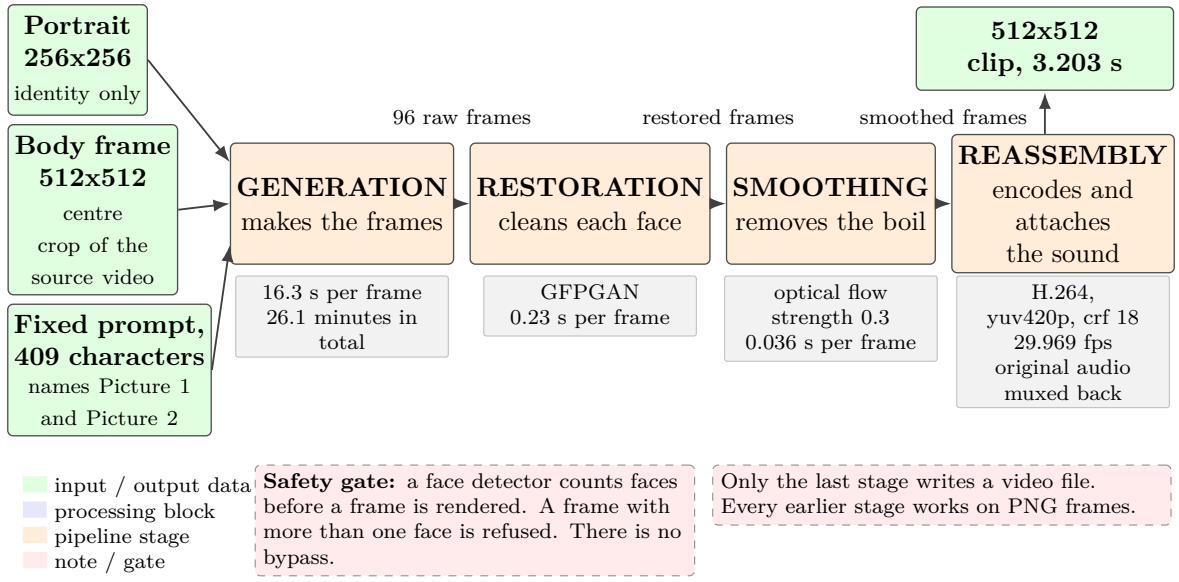


Figure 4: The four-stage pipeline: generation, restoration, smoothing and reassembly, with the cost measured for each stage.

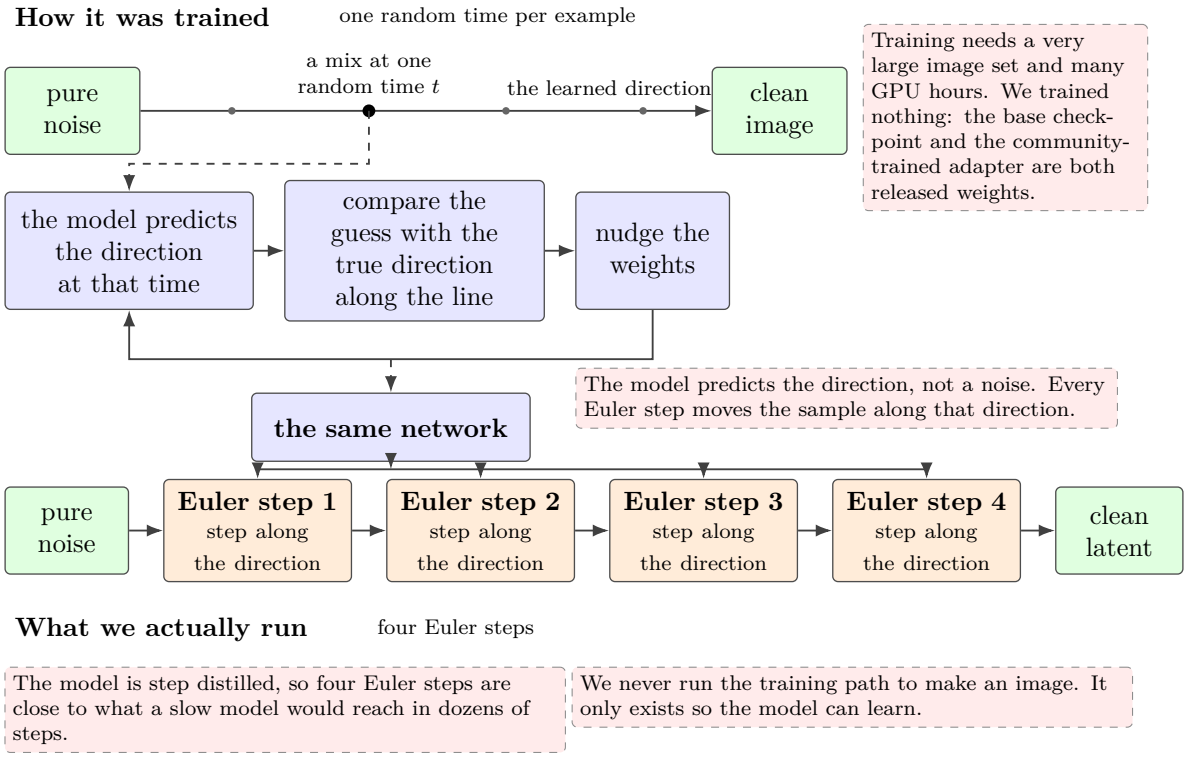


Figure 5: The rectified flow model in training and at sampling. Training draws one random time per example, mixes the image and noise at that time, and teaches the model the direction along the straight line between the two ends. Sampling starts from noise and takes four Euler steps along the predicted direction. Both jobs use one and the same network.

3.5 The model and the LoRA adapter

The model is a stack of transformer blocks (Figure 6). A latent token of 128 channels is projected to width 3072. It passes through five double-stream blocks, then 20 single-stream blocks and a final layer. In the first part the text and image tokens stay apart, and they meet through joint

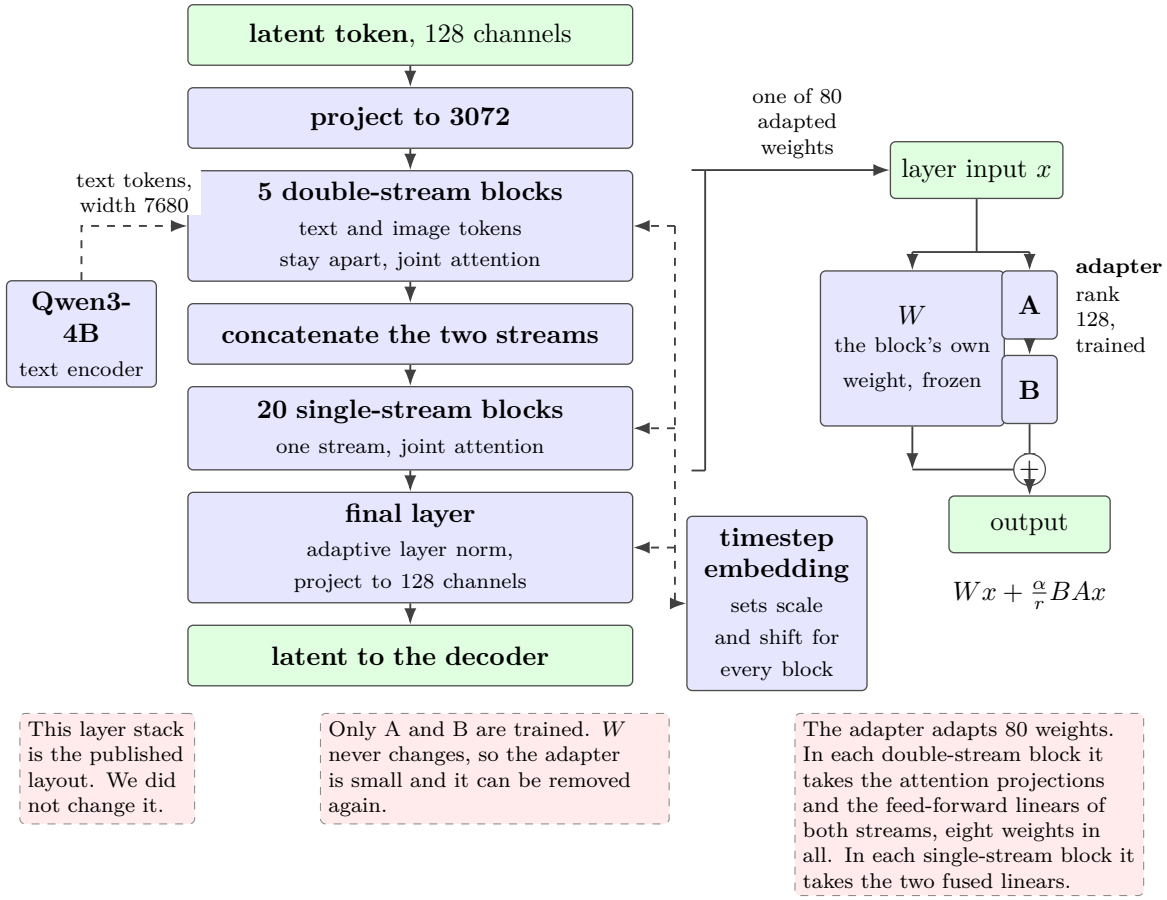


Figure 6: The model path: the latent token is projected to width 3072, passes the double-stream and single-stream blocks, and the final layer returns the latent for the decoder. The text encoder and the timestep embedding enter as conditioning. The bracket on the right marks one weight inside a block, where a small trained adapter sits beside the block's own frozen weight.

attention.

Later the two streams are joined into one. The final layer projects back to 128 channels for the decoder.

The adapter is a small pair of matrices, A and B, trained beside each frozen weight. The adapter attaches inside the transformer blocks. It takes the attention projections and the feed-forward linears, and in the single-stream blocks the two fused linears. The block output adds a scaled product of A and B to the frozen term. Only A and B are trained, so the base weights never change. The adapter carries the head swap behaviour.

3.6 Generation

Generation (Figure 7) covers conditioning, sampling and decoding. A fixed prompt of 409 characters names Picture 1 as the body and Picture 2 as the head.

Qwen3-4B encodes the prompt once per session on the CPU, about 80 seconds, and its memory is then freed. Both pictures become VAE latents: the 512x512 body crop a 64x64x32 grid of about 1024 tokens, the 256x256 portrait about 256 tokens.

FLUX.2-klein-4B renders each frame in four sampling steps. Each step moves the latent along the direction the model predicts. The sampler calls the model once per step and feeds the result back. The text embeddings and the two image latents guide every step. A community-trained LoRA adapter of 80 adapted weights carries the head swap.

The same VAE decodes the final latent into pixels. Picture order is part of the adapter

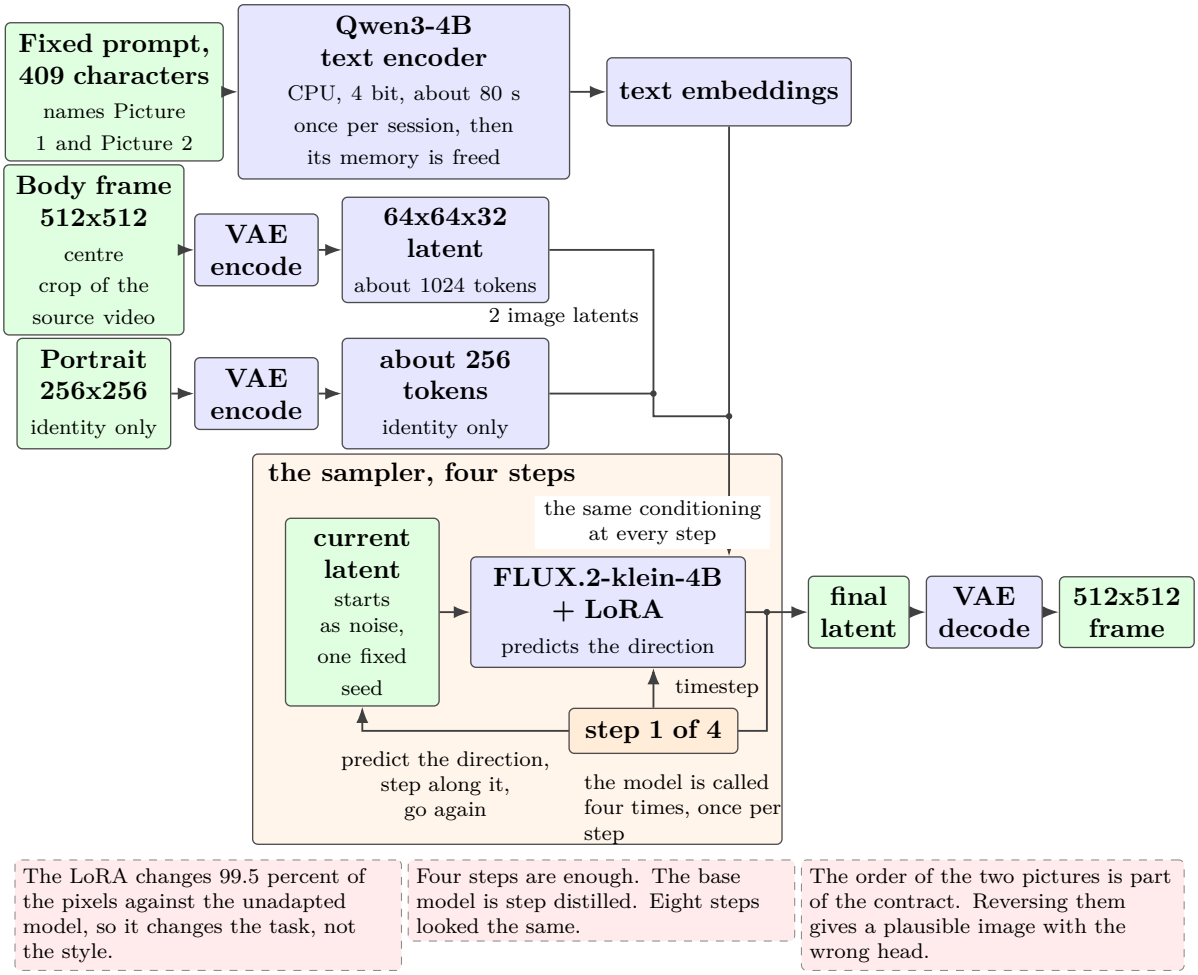


Figure 7: Inside the generation stage: Qwen3-4B encodes the prompt once, and the sampler then calls the model four times, once per step. Each call receives the current step number and the same conditioning, and the result is fed back into the loop. The last result is decoded into a 512x512 frame.

contract: the adapter's own documentation names Picture 1 the body and Picture 2 the head, and says not to reverse the images. A reversed order gives the wrong head.

The generator runs as a 4-bit GGUF file. A detector refuses a frame with more than one face, and the gate has no bypass.

The output is 96 frames from 5.8 to 9.0 seconds of the source, all with one fixed seed. Each frame took 16.3 seconds, so the stage used 26.1 minutes.

3.6.1 Adapter version comparison

The recorded clip uses adapter v1 throughout. A newer adapter, v1.1, was released during the work, and we tested two prompt variants, A and B. Table 4 compares the two versions.

v1.1 ran, but it went far past the memory of the card, which is why it slowed down so much. The v1.1 adapter has rank 512 against the 128 of v1, so it is four times larger. That is why it went past the card and slowed down. The extra time was not worth it for us.

We judged the look by eye and preferred v1, so the clip kept the v1 output. No metric script was run on v1.1, so every other number here comes from v1. The grids in Figure 8 are the only evidence.

	v1 (recorded)	v1.1 (new)
Adapter rank	128	512
Seconds per frame	16.34	about 199, twelve times slower
Peak VRAM allocated	3.60 GiB	4.68 GiB, past the 4.0 GiB of the card
Peak system RAM	5.48 GiB	5.3 to 5.5 GiB
Cost for 96 frames	26 minutes	about 5.3 hours

Table 4: Adapter v1 against v1.1 on the same machine; only v1 was measured with the metric script.

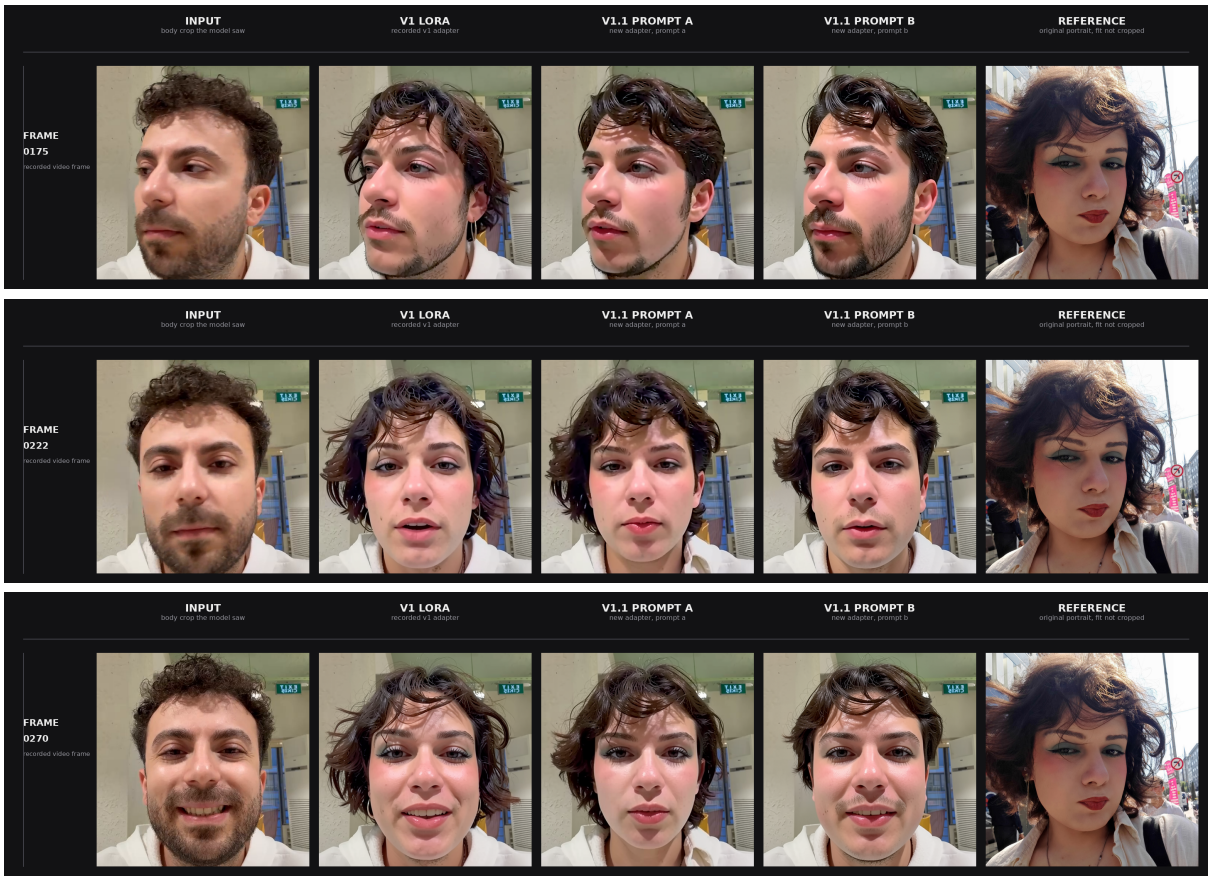


Figure 8: Each row is one source frame, the frame number printed inside. The five columns run from the input body crop through the v1 adapter, v1.1 with prompt A, v1.1 with prompt B, to the reference portrait; only the adapter version and the prompt variant change.

3.7 Face restoration

GFPGAN v1.4 cleans every frame (Figure 9). It detects and aligns the face, runs a restoration network, and composites the crop back. The weights are official, but we built the network in project code because the published wrappers do not import.

The stage costs 0.23 seconds per frame. The frames looked cleaner to us, and the mean face change came down, though the tail improved less.

Hair noise fell slightly, a small gain on its own. The fidelity weight does nothing: the v1.3.4 code passes no weight argument, and newer versions accept it without reading it. Restoration cannot remove flicker, because it sees one frame at a time.

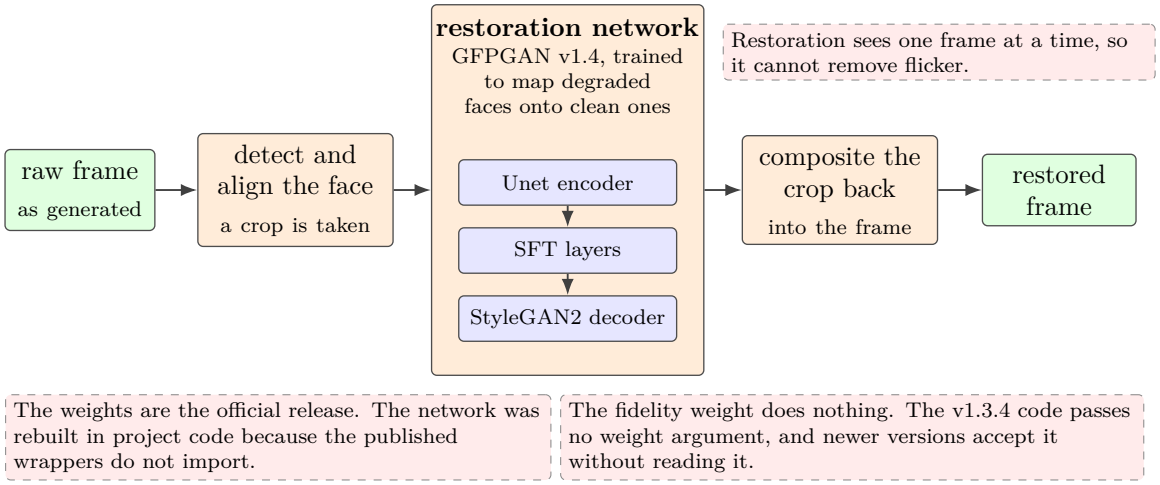


Figure 9: The face restoration stage. Each raw frame is taken on its own: the face is detected and aligned, a crop runs through the restoration network, and the restored crop is composited back into the frame.

Frames smoothed	Strength	Face change	Distance from the control	p95	Consecutive identity	Hair noise
Raw	0.3	6.9599	+0.3034	16.2944	0.9330	32.2358
Raw	0.6	6.6663	+0.0098	15.6581	0.9391	32.0603
Restored, then flow	0.3	6.6303	-0.0262	15.2802	0.9377	31.7306
Restored, then flow	0.6	6.3071	-0.3494	14.7575	0.9431	31.5448

Table 5: The smoothing sweep: two frame sets at two strengths, all measured on the same 96 frames.

3.8 Temporal smoothing

The smoothing stage compares frames with each other and removes the boil (Figure 10). The per-pixel deflicker averages each pixel with its temporal neighbours, and it is too strong. It costs 0.007 seconds per frame.

The deflicker pushes the face change below the unswapped level, so it removes real motion as well as the boil. Optical flow estimates the dense motion between neighbouring frames and warps them onto the current one. Because it follows the same surface point across time, a moving head does not smear. It costs 0.036 seconds per frame.

The control fixes the real motion level at 6.66, and the distance column is the face change minus that level. At strength 0.3 the chained pass comes close enough to the control, and the raw pass stays above it. A stronger setting sits below the control, which is over-smoothing.

The recommended output is the chained set at strength 0.3. It beats the flow-only variant on face change, on the tail and on hair noise. Only consecutive identity is slightly lower. The deflicker reaches its low face change only by destroying motion, so we kept flow.

3.9 Reassembly

The encoder turns a frame set into a playable clip (Figure 11). It writes H.264 at 29.969 fps, muxes the original audio back, and trims both streams. Its output is a 512x512 clip of 3.203 seconds.

The encode is lossy: it changes pixels slightly, but it cannot alter the face motion. In the raw swapped set the face boils, and the results table holds the values.

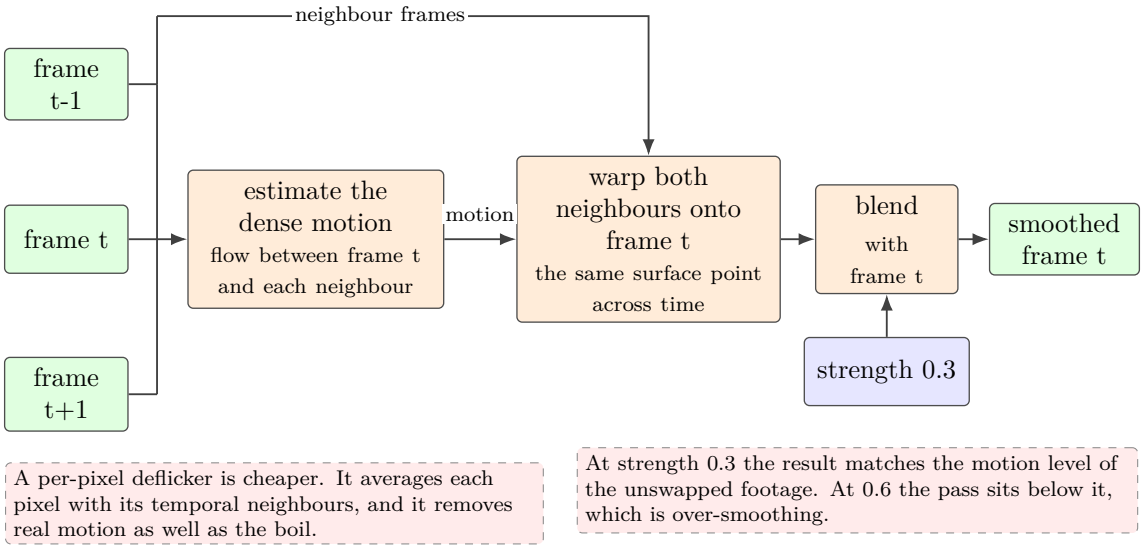


Figure 10: The temporal smoothing stage. Each frame is compared with its two neighbours: the dense motion is estimated, the neighbours are warped onto frame t and the three are blended at strength 0.3.

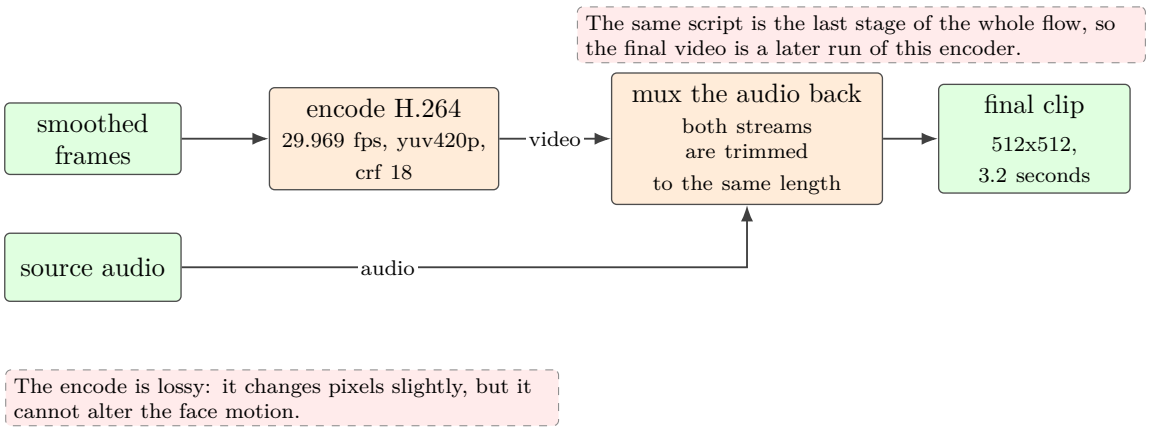


Figure 11: The reassembly stage. The smoothed frames are encoded as H.264 at the source rate and the original audio is muxed back; both streams are trimmed to the same length.

3.10 Results

All numbers come from one script, run once per variant, each on the same 96 frames (175 to 270). Lower hair noise is better, and higher identity values are better. Face change should sit near the control level, because a value below it means the smoother removed real motion. The control row is the unswapped source footage.

Identity to the portrait confirms the swap direction. The unswapped footage scores near zero, and every swapped variant scores positive. The recommended set held up well against the control on the mean values. Figure 12 shows the effect of each stage on one frame.

3.11 Problems and decisions

Attaching the adapter before the first forward pass cost 46 to 48 seconds per frame. Attaching it after one plain pass cost 16 seconds per frame. Peak VRAM was the same in both orders.

We standardised on the fast order. The internal cause was not established, but the effect reproduces exactly and explains an earlier timing discrepancy.

Stage	Face change (mean)	Face change (p95)	Consecutive identity	Identity to the portrait	Hair noise
Control (unswapped source footage)	6.6565	12.3488	0.9852	−0.0356	31.5999
Raw head swap	7.4510	17.3794	0.9228	0.2153	32.5703
Restoration	7.1602	16.2097	0.9273	0.2153	32.0721
Flow on the raw frames	6.6663	15.6581	0.9391	0.2148	32.0603
Restoration then flow, strength 0.3 (recommended)	6.6303	15.2802	0.9377	0.2152	31.7306
Deflicker (alternative)	5.6269	12.6650	0.9357	0.2106	31.8960

Table 6: All measured variants on the same 96 frames.



body input raw swap after restoration after smoothing

Figure 12: The same frame 0175 at four points of the flow; the four images are all 512x512.

One fixed seed covers the whole clip, so the noise input is constant. The boil still happens, because the model is sensitive to small input changes. Only a pass across frames reduces it.

3.12 Limits

The results table shows a mean at the control level, but the tail stays high, so the picture still jumped on some frames. Each step works inside a frame or a short window, so no step removed that tail.